Case Study: Re-architecting the 3rd Party Product Review System

Author: Ankul Choudhary **Stack proposed**: Java 21 + Spring Boot 3.x, Postgres, Redis, Kafka, OpenSearch, Kubernetes

---

Executive Summary

The ask. Replace a critical but outdated 3rd-party Product Review System with an in-house, modern alternative — without downtime, without data loss, and without disrupting the 10+ teams that depend on it today.

The proposal. A 5-month, ~6-FTE-average programme that delivers an in-house review platform on **Java 21 + Spring Boot 3.x**, backed by Postgres (primary + read replica), Redis, Kafka, and OpenSearch. Four services: a single `review-service` (writes + reads), a `moderation-service` (rule engine + ML pre-classifier + human queue), a transitional `vendor-adapter` for backwards compatibility at the legacy URL, and a `migration-worker` for backfill and reconciliation. The architecture is intentionally conventional — proven components in proven patterns — so the risk lives in the *migration*, not the *technology*.

Why this is safe. The cutover is a feature-flag flip, not a deploy. Migration runs in 6 reversible steps: snapshot import → dual-write → shadow-read parity → progressive ramp (1% → 100%) → vendor writes off → contract termination. Every step has a measurable exit gate. The ramp itself is guardrailed by 8 metrics (error rate, P99 latency, parity, conversion, SEO impressions) with **automatic rollback** on breach. Manual rollback target: < 60 seconds.

Why now. The current vendor causes 250 ms PDP latency (synchronous calls), blank-PDP outages when the vendor is down, fully-manual moderation, and per-call pricing that scales with traffic. We will reduce read P99 from ~250 ms to **< 50 ms**, GDPR turnaround from 7+ days to **< 24 hours**, and total cost to **≤ 60%** of today's spend.

What we ask of leadership. 1. Sponsor the 5-month commitment and the ~7.5-FTE peak resourcing in Phase 2–3. 2. Endorse three Go/No-Go gates — Build Complete (G1), Migration Ready (G2), Cutover Authorisation (G3). 3. Underwrite the comms plan to the 10+ downstream teams; vendor-adapter buys 2 quarters of grace, not unlimited time.

What we are not doing in Phase 1. No AI summaries, no multilingual moderation, no behavioural A/B testing, no active-active multi-region. Phase 2 starts only after hypercare exit (T+30d), with the AI summary epic as the leading candidate — unlocked by the data ownership this project delivers.

The bottom line. This is a high-confidence, low-novelty rebuild with a high-value migration. The plan is conservative on technology, aggressive on safety mechanisms, and explicit on the trade-offs.

---

1. Project Planning

1a. Detailed Project Plan

Scope

In scope (Phase 1 — replace the vendor) | Area | Included | |---|---| | Review lifecycle | Submit, edit, delete, moderate (approve / reject / flag), merchant reply, helpful / report votes | | Media | Image upload, image moderation, CDN delivery (video deferred) | | Display | PDP rating summary, review listing with sort/filter, schema.org markup for SEO | | Moderation | Rule engine + ML pre-classifier (profanity, spam, PII) + human moderation queue | | Search | Full-text + faceted filters (rating, verified buyer, with-photos, date) | | Migration | Full historical import from vendor, dual-write, shadow-read, ramp, decommission | | APIs | Public read APIs, authenticated write APIs, partner/legacy adapter at

vendor URL | | Admin | Moderation portal, audit log, bulk actions | | Analytics | CDC to data lake, daily metrics parity with vendor reports | | Compliance | GDPR delete, PII encryption, audit trail |

Out of scope (Phase 1 — explicit non-goals) - AI-generated review summaries / Q&A (Phase 2) - Multi-language moderation beyond English (Phase 2) - Loyalty / rewards integrations beyond what the vendor provides today - Multi-tenant SaaS-ification of the new system - Video reviews - Mobile SDK rewrite (mobile keeps calling the gateway)

Timeline (~5 months, 6 phases)

#	Phase	Weeks	Key Milestones	Exit Criteria
0	Discovery	1–2	Vendor contract & API audit, traffic profile, data dictionary, success KPIs signed off	HLD/LLD approved by Staff + Director
1	Foundation	3–5	Repos, CI/CD, infra (K8s, PG, Kafka, Redis, OpenSearch), schema v1, observability baseline	First service deployed to staging with green pipeline
2	Build	6–12	Core write/read services, moderation, vendor adapter, admin portal, contract tests	Feature-complete in staging behind flags
3	Migration	13–16	Bulk import, dual-write live, shadow-read parity ≥ 99.99%, delta sync stable	Reconciliation report signed off
4	Cutover	17–19	Traffic ramp 1% → 100% via flag, vendor read traffic stopped	100% traffic on new system, KPIs within SLO for 7 days
5	Hypercare & Decommission	20–22	Vendor write traffic stopped, contract terminated, retro, Phase 2 roadmap	Vendor invoice = $0, no P1 incidents for 30 days

Resources (~6 FTE average, ~7.5 peak)

Role	FTE	Active phases	Notes
Engineering Manager (you)	1.0	0–5	Delivery, stakeholder mgmt, runs cutover war-room
Staff Engineer / Architect	0.5	0–4	Heavy in 0–1, design reviews + escalation thereafter
Backend Engineer (Sr)	3.0	1–5	Parallelize across write / read / moderation
Frontend Engineer	1.0	2–5	Admin moderation portal; storefront PDP is a small slot
Data Engineer	0.5	2–4	Migration ETL, reconciliation, CDC; heavy in Phase 3
SRE	0.5	1–5	Heavy at infra setup + cutover, light otherwise
QA / SDET	1.0	2–5	Contract, parity, and load test automation
Security Engineer	0.25	1–4	Threat model, gate reviews, GDPR sign-off
Product Manager	0.25	0–5	Scope, KPIs, stakeholder asks
Product Designer	0.25	1–3	Admin portal UX
TPM	0.25	3–4	Cross-team coordination during migration & cutover
Total avg	**~6 FTE**		**~7.5 FTE peak in Phase 2–3**

Total effort: ~30 person-months.

Risks (register)

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	Data loss / corruption during migration	Med	High	Idempotent backfill, checksum reconciliation, vendor snapshot retained 90 days post-cutover	Data Eng

#	Risk	Likelihood	Impact	Mitigation	Owner
R2	SEO regression (review-rich snippets in SERP)	Med	High	Preserve URLs + schema.org markup; structured-data parity tests; staged ramp watching impressions	FE + SEO
R3	PDP latency regression at peak	Med	High	Redis-cached aggregates, shadow load test at 5× peak, gradual ramp with auto-rollback	SRE
R4	Vendor API throttle/outage during dual-run	High	Med	Adapter with Resilience4j circuit breaker; fallback to cached aggregates	BE
R5	Spam/abuse spike post-launch (no vendor blocklist)	High	Med	Export vendor blocklist pre-cutover; velocity rate limits; human-in-loop moderation	BE + Trust
R6	10+ downstream teams calling vendor APIs	High	High	Adapter exposed at vendor URL for backwards compat; deprecate over 2 quarters with comms plan	EM + TPM
R7	GDPR / privacy gap on PII handling	Low	High	Threat model in Phase 1; encryption at rest; tombstone-on-delete event; legal sign-off pre-cutover	Security
R8	Team ramp on Java 21 virtual threads / Spring Boot 3	Med	Med	Tech-spike week in Phase 1; pair programming; lint rules for `synchronized` on virtual threads	Architect
R9	Vendor contract tail (auto-renewal)	Low	Med	Notice served before Phase 0 end; sunset date locked into timeline	EM + Legal
R10	Scope creep (Phase 2 features pulled into Phase 1)	High	Med	Explicit non-goals in scope; change-control via EM	EM

1b. Stakeholders — RACI

Workstreams: **WS1** Architecture & Design · **WS2** Build · **WS3** Migration · **WS4** Cutover · **WS5** Comms & Change Mgmt · **WS6** Decommission & Contract

Stakeholder	WS1 Design	WS2 Build	WS3 Migration	WS4 Cutover	WS5 Comms	WS6 Decommission
Engineering Manager (you)	A	A	A	A	A	A
Staff Engineer / Architect	R	C	C	C	I	I
TPM	I	I	R	R	R	R
Backend Team	R	R	R	R	I	C
Frontend Team	C	R	I	R	I	I
Data Engineer	C	C	R	C	I	C
SRE	C	R	C	R	I	R
QA / SDET	C	R	R	R	I	I
Security Engineer	C	C	C	C	I	C
Product Manager	C	C	C	C	R	C
Customer Support Lead	I	I	I	C	C	I
Marketing / SEO Lead	I	I	I	C	C	I

Stakeholder	WS1 Design	WS2 Build	WS3 Migration	WS4 Cutover	WS5 Comms	WS6 Decommission
Legal / Privacy	C	I	C	I	I	C
Vendor Account Manager	I	I	C	C	I	R
Executive Sponsor (VP Eng)	I	I	I	A	I	A

R = Responsible · A = Accountable · C = Consulted · I = Informed

---

1c. Communication Plan

Audience	Format	Cadence	Owner	Channel
Executive sponsor	RAG status + 3 bullets (risks, decisions, asks)	Weekly	EM	Email + Slack DM
Cross-team stakeholders (Product, CS, Marketing, Partner teams)	Demo + roadmap update	Bi-weekly	EM + PM	30-min Zoom + recording
Engineering team	Stand-up	Daily	Tech lead	Slack huddle
Engineering team	Sprint planning / review / retro	Bi-weekly	EM	Zoom
Architecture decisions	ADR in repo	As needed	Author	`docs/adr/*.md`
Risk register	Living doc	Reviewed weekly	EM	Confluence
Migration cutover	Go/No-Go meeting + runbook walkthrough	T-7d, T-1d, T-0	EM + SRE	Zoom + war-room Slack
Cutover war-room	Live updates	Hourly during ramp	SRE on-call	`#review-cutover` Slack
Customer Support	FAQ + escalation path	T-7d, refreshed weekly during hypercare	PM	Confluence + CS Slack
Downstream API consumers	Deprecation notice + migration guide	T-30d, T-14d, T-1d	TPM	Email + dev-portal
Post-launch KPI review	Metrics readout vs. targets	Weekly for 4 weeks, then monthly	EM + PM	Email + dashboard link
Project retro	Written retro + action items	T+30d	EM	Confluence

---

2. High-Level Design and Low-Level Design

2.0 Goals & Non-Functional Requirements

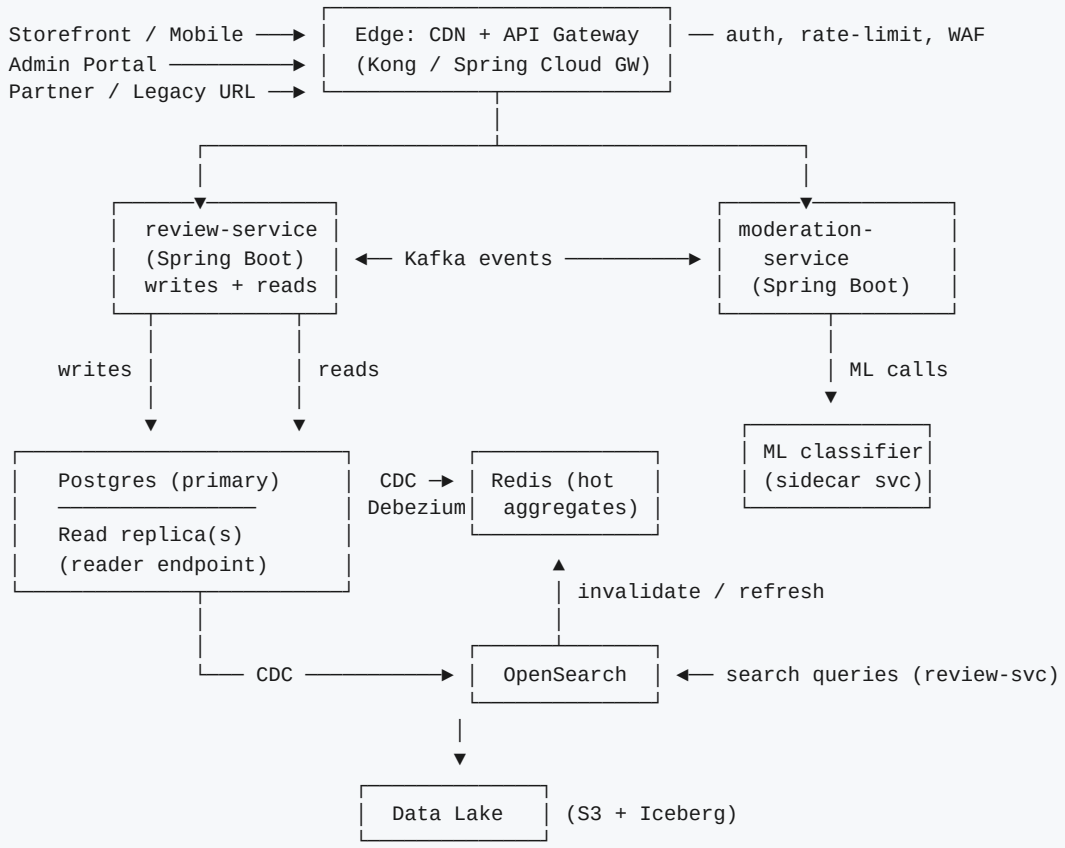
Dimension	Target
Read P99 latency (PDP rating + listing)	< 50 ms server-side
Write P99 latency (submit review)	< 200 ms
Availability	99.95% (multi-AZ, graceful degradation on dependency loss)

Dimension	Target
Throughput headroom	5× current peak (currently ~2k reads/sec, ~50 writes/sec)
RPO / RTO	RPO 5 min · RTO 30 min
Data parity at cutover	≥ 99.99% per reconciliation report
Moderation SLA	Auto-classified < 2 sec; human queue < 4 hours business
Cost	≤ 60% of current vendor + integration spend at steady state

2.1 Pain Points in the Existing (Vendor) Architecture

#	Pain point	Root cause
P1	PDP latency spikes	Synchronous vendor REST call on every product view
P2	Vendor outages → "no reviews" placeholder on PDP	No local cache or fallback
P3	Cannot add fields (e.g., size-fit, verified-buyer flag)	Vendor's fixed schema
P4	Moderation is fully manual & email-based	No rule engine, no ML pre-filter
P5	No faceted filtering ("with photos", "verified", rating filter)	Vendor API doesn't expose facets
P6	Reporting lags 24h, doesn't match our dimensions	One-way CSV export from vendor
P7	Per-call pricing scales with traffic	Vendor commercial model
P8	10+ teams call vendor APIs directly	No internal abstraction
P9	GDPR delete takes 7+ days	Manual vendor ticket process
P10	No idempotency on submission	Vendor swallows duplicates silently

2.2 Proposed High-Level Architecture



Service inventory

Service	Spring Boot module	Responsibility	Scaling
`review-service`	`services/review`	Submit / edit / delete, vote, reply, plus PDP summary, listing, search; one domain, one repo	HPA on RPS; virtual threads; reads routed to PG read replica via `@Transactional(readOnly=true)`
`moderation-service`	`services/moderation`	Rule engine + ML calls + human-queue API; Kafka-consumer workload with independent release cadence	HPA on Kafka lag
`vendor-adapter`	`services/adapter`	Translates legacy vendor URL → new APIs (transitional)	Fixed 2 replicas
`migration-worker`	`services/migration`	Bulk import + delta sync + reconciliation jobs	Job-based (K8s CronJob + workers)
`admin-portal` (FE)	`apps/admin`	Moderation UI for ops team	Static + BFF

2.3 Design Decisions (ADRs)

ADR	Decision	Alternatives	Rationale
ADR-01	**Postgres** as source of truth	DynamoDB, Mongo	Transactional moderation state, joins for admin, mature operator skills, JSONB for flexible metadata; avg row size and access pattern fit OLTP comfortably
ADR-02	**Redis** for hot rating aggregates	DB-only with materialized views	PDP read is read-heavy, latency-critical; Redis gives sub-ms p99; invalidation via Kafka consumer keeps it correct
ADR-03	**OpenSearch** for review listing & search	Postgres FTS, Elasticsearch	Faceted filters at scale; Elasticsearch licensing pushed us off; OpenSearch is API-compatible and AWS-managed
ADR-04	**Kafka** as backbone	RabbitMQ, AWS SQS+SNS	Replay capability for reconciliation, ordering per partition (per `product_id`), CDC integration via Debezium
ADR-05	**Spring Web MVC on virtual threads** (not WebFlux)	WebFlux	JDK 21 makes blocking I/O cheap; keeps stack imperative & debuggable; team is faster on MVC; throughput parity for our workload
ADR-06	**Spring Data JPA + Hibernate 6** for writes; **JdbcTemplate** for hot read paths	JPA everywhere, MyBatis	JPA's productivity for the moderation domain; raw JDBC where every microsecond counts on PDP reads
ADR-07	**Resilience4j** circuit breaker on the vendor adapter	Hystrix (deprecated), Spring Retry alone	Modern, virtual-thread friendly, integrates with Micrometer
ADR-08	**Flyway** for schema migrations	Liquibase	Plain SQL, simpler review story, tighter Spring Boot integration
ADR-09	**Adapter at vendor URL** during migration	Big-bang cutover	10+ downstream teams cannot all migrate simultaneously; adapter buys 2 quarters of decoupling
ADR-10	**Outbox pattern** for write→event publish	Direct Kafka publish from controller	Atomic with DB transaction; eliminates "DB committed but event lost"
ADR-11	**Event-sourced moderation** (state transitions as events)	Mutable status column only	Auditability, replay during incident, easy admin timeline view
ADR-12	**Feature flags** (LaunchDarkly or OpenFeature) for cutover	Config files	Per-percentage ramp, instant rollback, per-tenant overrides for canaries
ADR-13	**Single `review-service`** (writes + reads), not split CQRS services	Separate write-service and read-service	At our scale (~2k r/s, ~50 w/s) splitting is premature CQRS. Read replica + Redis cover latency and scale at the data tier. Single repo, single team, single deploy. If JVM contention ever becomes real, the same image can be deployed as two K8s Deployments (reader vs writer) — an ops decision, not an architecture split

2.4 Database & Storage Choices Summary

Store	Purpose	Why
Postgres 15 (Aurora / RDS)	Source of truth: reviews, votes, replies, moderation events	Transactional, joinable, mature; expand-contract migrations safe

Store	Purpose	Why
Redis 7 (cluster mode)	Rating aggregates per product, idempotency keys, rate limits	Sub-ms latency on hot path
OpenSearch	Full-text + faceted search of approved reviews	Facets, relevance, scale
S3 + Iceberg	Data lake for analytics, ML training, vendor snapshot retention	Cheap, queryable via Athena
S3 + CloudFront	Review media (images)	CDN-fronted blob store
Kafka	Event log: `review.events`, `moderation.events`, `audit.events`, `cdc.review`	Replay, fan-out, ordering per `product_id`

2.5 Entity & Schema Design

```
-- Source-of-truth schema (Postgres). All FK constraints elided for brevity.

CREATE TYPE review_status   AS ENUM ('pending','approved','rejected','flagged','deleted');
CREATE TYPE review_source   AS ENUM ('native','migrated_vendor');
CREATE TYPE vote_kind       AS ENUM ('helpful','report');
CREATE TYPE moderation_actor AS ENUM ('system_rule','ml_classifier','human','migration');

CREATE TABLE review (
    id              UUID        PRIMARY KEY,
    product_id      UUID        NOT NULL,
    user_id         UUID        NOT NULL,
    order_id        UUID        NULL,                -- enables verified-buyer flag
    rating          SMALLINT    NOT NULL CHECK (rating BETWEEN 1 AND 5),
    title           TEXT        NULL,
    body            TEXT        NOT NULL,
    language        VARCHAR(8)  NOT NULL DEFAULT 'en',
    status          review_status NOT NULL DEFAULT 'pending',
    source          review_source NOT NULL DEFAULT 'native',
    legacy_id       TEXT        NULL,                -- vendor id, for reconciliation
    metadata        JSONB       NOT NULL DEFAULT '{}',
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    moderated_at    TIMESTAMPTZ NULL,
    deleted_at      TIMESTAMPTZ NULL,
    UNIQUE (user_id, product_id, order_id)           -- 1 review per user per purchased item
);
CREATE INDEX review_pdp_idx       ON review (product_id, status, created_at DESC) WHERE deleted_at IS NULL;
CREATE INDEX review_user_idx      ON review (user_id, created_at DESC);
CREATE INDEX review_legacy_id_idx ON review (legacy_id) WHERE legacy_id IS NOT NULL;

CREATE TABLE review_media (
    id          UUID PRIMARY KEY,
    review_id   UUID NOT NULL REFERENCES review(id) ON DELETE CASCADE,
    url         TEXT NOT NULL,
    media_type  VARCHAR(16) NOT NULL,
    position    SMALLINT NOT NULL,
    moderation_status review_status NOT NULL DEFAULT 'pending'
);

CREATE TABLE review_vote (
    review_id UUID NOT NULL REFERENCES review(id) ON DELETE CASCADE,
    user_id   UUID NOT NULL,
    kind      vote_kind NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (review_id, user_id, kind)
);

CREATE TABLE review_reply (
    id          UUID PRIMARY KEY,
    review_id   UUID NOT NULL REFERENCES review(id) ON DELETE CASCADE,
    author_type VARCHAR(16) NOT NULL,        -- 'merchant' | 'admin'
    author_id   UUID NOT NULL,
    body        TEXT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE moderation_event (
    id           BIGSERIAL PRIMARY KEY,
    review_id    UUID NOT NULL REFERENCES review(id) ON DELETE CASCADE,
    actor        moderation_actor NOT NULL,
    actor_id     TEXT NULL,                  -- rule id, model version, or human user id
    from_status  review_status NULL,
    to_status    review_status NOT NULL,
    reason_code  VARCHAR(64) NULL,
    notes        TEXT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX moderation_review_idx ON moderation_event (review_id, created_at);
```

```
CREATE TABLE product_rating_summary (
    product_id   UUID PRIMARY KEY,
    avg_rating   NUMERIC(3,2) NOT NULL,
    review_count INTEGER NOT NULL,
    histogram    JSONB    NOT NULL,          -- {"1":3,"2":4,"3":10,"4":80,"5":200}
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Outbox table for transactional event publishing
CREATE TABLE outbox_event (
    id          UUID PRIMARY KEY,
    aggregate   VARCHAR(64) NOT NULL,        -- 'review' | 'moderation' | 'reply'
    aggregate_id UUID NOT NULL,
    type        VARCHAR(64) NOT NULL,        -- 'ReviewCreated', etc.
    payload     JSONB NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at TIMESTAMPTZ NULL
);
CREATE INDEX outbox_unpublished_idx ON outbox_event (created_at) WHERE published_at IS NULL;

-- Idempotency table (or Redis equivalent)
CREATE TABLE idempotency_key (
    key         TEXT PRIMARY KEY,
    user_id     UUID NOT NULL,
    request_hash TEXT NOT NULL,
    response    JSONB NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

2.6 API Design

REST + JSON, versioned `/v1`, OpenAPI spec generated via `springdoc-openapi`. Auth: OAuth2 JWT (resource server). All write endpoints require `Idempotency-Key` header.

Method	Path	Auth	Purpose	Notes
POST	`/v1/reviews`	user	Submit a review	Idempotency-Key required; returns 202 if pending moderation
PATCH	`/v1/reviews/{id}`	user (owner)	Edit text/title within edit window	24h edit window; resets to `pending`
DELETE	`/v1/reviews/{id}`	user (owner)	Soft-delete user's review	Tombstone, cascades to GDPR delete
GET	`/v1/products/{productId}/reviews`	public	List reviews, sort+filter+facets	Cursor pagination; cached at edge
GET	`/v1/products/{productId}/rating-summary`	public	Avg, count, histogram	Redis-backed; CDN TTL 60s SWR 600s
POST	`/v1/reviews/{id}/votes`	user	Helpful or report vote	Idempotent on (user, review, kind)
POST	`/v1/reviews/{id}/replies`	merchant	Merchant reply	Single reply per review
GET	`/v1/admin/reviews`	admin	Moderation queue with filters	Auth: `role=moderator`
PATCH	`/v1/admin/reviews/{id}/moderation`	admin	Approve / reject / flag	Emits moderation event
POST	`/v1/admin/users/{id}/gdpr-delete`	admin	Trigger GDPR tombstone	Emits `UserGdprRequested`

Method	Path	Auth	Purpose	Notes
POST	`/internal/migration/import`	service	Bulk import endpoint	`source=migrated_vendor`; auth via service-mesh mTLS
GET	`/internal/health`, `/actuator/*`	infra	Liveness, readiness, metrics	Spring Actuator

Key API conventions - **Pagination**: opaque base64 cursor, `?cursor=…&limit=20`. Default sort: `helpful_desc, created_desc`. - **Errors**: RFC 7807 problem+json with `type`, `title`, `status`, `detail`, `traceId`. - **Idempotency**: `Idempotency-Key` header; server stores hash + response for 24h. - **Backwards compat**: vendor URLs proxied by `vendor-adapter`; same response schema, different backend.

2.7 Service-Level LLD

review-service

- **Stack**: Spring Boot 3, Spring Web MVC on virtual threads, Spring Data JPA + JdbcTemplate (hot read paths), spring-kafka, Lettuce (Redis), OpenSearch Java client, Resilience4j.
- **Datasource topology**:
- Two `DataSource` beans wired via `AbstractRoutingDataSource`: **primary** (writer) and **replica** (Aurora reader endpoint or PG streaming replica).
- Routing key derived from `TransactionSynchronizationManager.isCurrentTransactionReadOnly()`, so `@Transactional(readOnly=true)` automatically lands on the replica.
- HikariCP pools sized differently per role.
- **Write components**:
- `ReviewController` (POST/PATCH/DELETE) — request validation, idempotency check.
- `ReviewCommandService` — transactional: persist review + outbox event in one TX.
- `OutboxRelay` — scheduled poller (or Debezium on `outbox_event`) → Kafka.
- `IdempotencyService` — Redis-backed; falls back to PG `idempotency_key` table.
- `ProfanityPreCheck` — fast local rule check before persisting (rejects obvious abuse synchronously).
- **Read components**:
- `ProductRatingController` → `RatingSummaryService` → Redis (hot) → JdbcTemplate on `product_rating_summary` (replica) on miss.
- `ReviewListingController` → OpenSearch Java client for facets/sort/filter.
- `RatingCacheConsumer` — Kafka consumer on `review.events`; on `ReviewApproved` / `ReviewRejected`, invalidates Redis aggregate and triggers OpenSearch index update.
- Caching: 60s edge TTL + 600s stale-while-revalidate; ETag on summary.
- Degraded mode: if OpenSearch is down, fall through to PG replica with simpler ordering and a `degraded=true` response header.
- **Concurrency**: virtual-thread executor for inbound HTTP; `@Transactional` boundaries kept short; bulk operations use platform threads.
- **Failure modes**: primary DB down → 503 on writes, reads continue from replica + Redis; Kafka down → outbox absorbs (eventual publish); Redis down → reads fall through to replica.
- **Ops escape hatch**: if read traffic ever causes JVM contention with writes, the same Docker image can be deployed as two K8s Deployments (`review-service-reader` with high replica count + tuned heap, `review-service-writer` with smaller fleet) — same code, different scaling profile. Not needed at launch.

moderation-service

- **Stack**: Spring Boot 3, spring-kafka, Resilience4j, ML client (gRPC).

- **Flow**: consumes `ReviewCreated` → calls ML classifier → applies rule engine → either auto-approves (publishes `ReviewApproved`) or enqueues for human review (`ReviewFlagged`).
- **Rule engine**: declarative YAML rules (loaded at startup, hot-reload on signal). Examples: `rating==1 AND length<10 → flag`, `contains_pii → reject`.
- **Human queue**: paginated via admin API; assignments tracked; SLA timer per item.

vendor-adapter (transitional)

- **Stack**: Spring Boot 3, WebClient (for outbound to vendor), Resilience4j (CB + retry + bulkhead).
- **Mode** (controlled by feature flag per route):
- `vendor_only` (initial)
- `dual_write` (writes go to both, reads still vendor)
- `shadow_read` (read both, return vendor, log diff)
- `new_primary` (read new, fall back to vendor on miss)
- `new_only` (vendor disabled)
- **Deprecation timeline**: kept for 2 quarters post-cutover; sunset announcement to API consumers.

migration-worker

- **Stack**: Spring Boot 3, Spring Batch for bulk, Kafka consumer for delta.
- **Jobs**:
- `bulk-import` — paginated vendor export → transform → upsert with `source=migrated_vendor`.
- `delta-sync` — vendor webhook / scheduled poll → upsert.
- `reconciliation` — counts and checksums per product window vs. vendor snapshot; produces report.

2.8 Sequence Diagrams (key flows)

Submit review (write path)

```
Client → Gateway → review-svc:
   1. validate JWT, schema, rating range
   2. check idempotency key
   3. profanity pre-check (sync, cheap)
   4. BEGIN TX
        INSERT review (status=pending)
        INSERT outbox_event (ReviewCreated)
      COMMIT
   5. respond 202 Accepted
   ─── async ───
   OutboxRelay → Kafka (review.events / ReviewCreated)
   moderation-svc consumes:
        ML classify → rules → decide
        emit ReviewApproved | ReviewFlagged
   review-svc consumers:
        on ReviewApproved → invalidate Redis aggregate, index in OpenSearch
        on ReviewApproved → recompute product_rating_summary
```

PDP rating summary (read path)

```
Client → CDN (60s TTL, SWR 600s) → Gateway → review-svc:
   Redis GET rating:{productId}
     hit  → return
     miss → JdbcTemplate fetch from product_rating_summary
            → SETEX rating:{productId} ttl=120s
            → return
   degraded if Redis+DB both fail → return last-good from CDN, header `degraded=true`
```

Moderator approves a flagged review

```
Admin UI → admin-API → review-svc:
   PATCH /admin/reviews/{id}/moderation {to: approved}
   BEGIN TX
     UPDATE review SET status=approved, moderated_at=now()
     INSERT moderation_event (...)
     INSERT outbox_event (ReviewApproved)
   COMMIT
   ─── async ───
   downstream consumers update Redis, OpenSearch, audit log
```

2.9 Old → New: Bottlenecks Removed

Legacy issue (from §2.1)	New design fix	Component
P1 sync vendor call on every PDP load	Local Redis aggregate, P99 < 5 ms	Redis + review-svc
P2 vendor outage = blank PDP	Multi-AZ Postgres, Redis fallback to DB, CDN stale-while-revalidate	All read tier
P3 fixed schema	Owned PG schema + JSONB `metadata` for additive fields	Postgres
P4 fully manual moderation	Rule engine + ML pre-classifier + human queue with SLA	moderation-svc
P5 no faceted filtering	OpenSearch facets	OpenSearch + review-svc
P6 reporting lag, dimension mismatch	Debezium CDC → S3/Iceberg → Athena, daily DAGs	Data lake
P7 per-call vendor pricing	Owned infra; cost scales sublinearly with traffic	Infra
P8 10+ teams call vendor directly	Vendor-adapter at legacy URL → new APIs; deprecate over 2 quarters	vendor-adapter
P9 GDPR delete takes 7+ days	Tombstone event; downstream consumers honor within 24h	Outbox + consumers
P10 silent duplicate submissions	`Idempotency-Key` + unique constraint `(user, product, order)`	review-svc

2.10 Cross-Cutting Concerns

- **Observability**: Micrometer → Prometheus; OpenTelemetry traces (trace context propagated through Kafka headers); structured JSON logs with `traceId`, `userId`, `productId`. Dashboards: RED per endpoint, business KPIs (reviews/day, approval rate, moderation SLA, parity %), Kafka lag.
- **Security**: OAuth2 JWT at gateway; admin endpoints require `role=moderator|admin`; PII (user_id, IP) encrypted at rest via PG TDE + column-level pgcrypto for free-text PII flagged by ML; secrets in HashiCorp Vault / AWS Secrets Manager; mTLS in service mesh.
- **Resilience**: Resilience4j circuit breaker on every outbound call (vendor, ML, OpenSearch); Kafka consumer DLQs with replay tooling; chaos drills before cutover (kill Redis, kill an AZ, throttle vendor).
- **Compliance**: GDPR tombstone within 24h; full audit log via `moderation_event` + `audit.events` topic retained 7 years in S3.
- **Cost controls**: Reserved instances for steady-state PG/OpenSearch; spot for migration workers; Kafka tiered storage for long retention.

---

3. Code Review

> Per the assignment, this is a **live discussion** on a piece of code provided during the interview. AI tools and IDE are permitted. There is no written deliverable in advance; the section below is the **approach and checklist I'll bring into the live session**.

3.1 How I'll run the review

1. **Frame before reading**. Ask 3 questions up-front: *what does this code do, who calls it, what's the failure mode that prompted the review?* — this anchors me on the contract rather than on syntax.
2. **Read top-down twice**. First pass for shape (entry points, dependencies, data flow). Second pass for correctness, line by line.
3. **Narrate aloud**. Verbalize hypotheses, name what I'm looking for, and flag uncertainty explicitly ("I'm not sure how this behaves under concurrent calls — let me check").
4. **Use AI tools transparently**. Cursor / Claude / Copilot used to (a) cross-check my interpretation of unfamiliar APIs, (b) generate hypothesis test cases, (c) suggest refactors I then evaluate. I'll show, not hide, the tool use — and I'll always confirm AI suggestions against the code itself.
5. **Separate severities**. Tag every finding as **Blocker / Major / Minor / Nit / Praise**. Don't let nits drown out the real issues.
6. **Lead with the contract, not the code**. If the function's contract is wrong, the implementation doesn't matter.
7. **Suggest the minimal fix, then the ideal one**. Respect the author's scope; offer the larger refactor as a follow-up.
8. **Note what's good**. Calling out good patterns is part of senior review — it teaches and it calibrates.

3.2 Review checklist (mental model I'll apply)

Correctness - Does the code match its stated contract / Javadoc / API spec? - Off-by-one, null handling, empty-collection, boundary values. - Time zones, locale, character encoding. - Floating-point / money handled with `BigDecimal`? - Error paths return correct types and codes (RFC 7807?).

Concurrency & Java 21 specifics - Thread safety of shared state; any `static` mutable field? - Use of `synchronized` blocks while running on virtual threads — does it pin? Prefer `ReentrantLock`. - `ThreadLocal` use under virtual threads (high cardinality risk). - Correct use of `CompletableFuture` / structured concurrency / `ExecutorService` shutdown. - Records — are they used as value objects only? No leaking mutable collections inside. - Sealed interfaces for closed hierarchies (events, results) instead of enums + visitor. - Pattern matching in `switch` — exhaustiveness?

Spring Boot idioms - Constructor injection (no field `@Autowired`). - `@Transactional` boundary correctness — `readOnly=true` on read paths; no calls across `@Transactional` from same class (self-invocation). - Proper layering: controller → service → repository; no JPA entities leaked to the API. - Bean validation (`@Valid`, `@NotNull`, `@Size`) on inputs. - Exception handling via `@RestControllerAdvice`, not try/catch in controllers. - DTOs as records; entities not serialized to JSON.

Persistence - N+1 query (lazy associations in a loop)? - Missing index for the access pattern? - Pagination strategy (cursor vs. offset) appropriate for scale? - `@Transactional` covers all writes that must be atomic, including outbox event. - Long-running transactions / SELECT FOR UPDATE held too long.

Security - AuthN and AuthZ at the right layer; ownership checks for `PATCH/DELETE` on user-owned resources. - SQL/command injection (parameterized queries, no string concat). - PII handling — logged? returned in errors? indexed by external systems? - Secrets in code or config? - Open redirects, SSRF on outbound calls. - CSRF for cookie-auth endpoints; CORS scoped correctly.

Performance - Hot loops doing I/O (DB call inside `for`). - Unbounded collections / pagination. - Excessive object allocation in hot paths. - Missing cache or wrong TTL. - Logging at INFO inside hot loop.

Resilience - Outbound calls have timeout + circuit breaker + retry with backoff? - Idempotency keys honored? - Failure modes graceful or do they cascade? - Dead-letter handling for Kafka consumers.

Observability - Structured logs with `traceId`, `userId`, `productId`. - Meaningful metric names; counters vs. histograms used correctly. - Errors logged once, at the right level, without leaking PII.

Testability & tests - Pure functions where possible; side-effects pushed to edges. - Tests cover the happy path **and** at least one failure mode. - No production code paths gated solely by mocks. - Testcontainers (PG/Kafka/Redis) over H2 for integration tests.

Readability - Naming reflects intent (no `data`, `manager`, `helper` without qualifier). - Functions short, single responsibility, low cyclomatic complexity. - Magic numbers extracted to named constants. - Comments explain *why*, not *what*. - Dead code removed.

API hygiene - Backwards compatibility (additive changes, no breaking renames). - Idempotency on writes. - Pagination + sort defined. - Errors typed and machine-readable. - Versioning strategy honored.

3.3 Output format I'll produce in the session

- A short verbal summary: "Overall: ship-with-changes / needs-rework / good-to-merge."
- 3–7 written findings, each with: **severity · file:line · what's wrong · why it matters · suggested fix**.
- 1–2 explicit calls of *what's good*.
- A list of follow-ups (out of scope for this PR but worth tracking).

---

4. Execution

4a. Task Breakdown and Assignment

Work is decomposed into **8 epics** mapped to the 6 timeline phases. Each epic contains 5–15 stories sized 1–3 days. Owners listed are the *primary* driver; pairs are encouraged.

#	Epic	Phase	Lead	Key stories (sample)	Dependencies
E1	Discovery & Design	0	Architect	Vendor API audit, traffic profile capture, data dictionary, KPI sign-off, HLD/LLD, ADRs, threat model	—
E2	Platform & Infra	1	SRE	K8s namespace, PG primary+replica, Redis cluster, Kafka, OpenSearch, CI/CD pipelines, observability stack, secrets vault	E1
E3	Domain Core (`review-service`)	2	BE Lead	Schema (Flyway), JPA entities, write API, idempotency, outbox, read API (summary, listing), Redis cache invalidator, OpenSearch indexer, contract tests	E2
E4	Moderation	2	BE #2	Rule engine YAML + loader, ML client (gRPC), Kafka consumer + DLQ, human queue API, audit events	E3 (events contract)
E5	Admin Portal	2–3	FE Lead	Moderation queue UI, bulk actions, audit timeline, GDPR delete UI, role-based access	E4 (admin API)
E6	Vendor Adapter	2–4	BE #3	Legacy URL adapter, dual-write coordinator, shadow-read with diff logger, mode flag (`vendor_only` → `new_only`), Resilience4j config	E3
E7	Migration & Backfill	3	Data Eng	Bulk import job (Spring Batch), delta sync (webhook + scheduled poll), reconciliation report, blocklist export, vendor snapshot to S3	E3, E6

#	Epic	Phase	Lead	Key stories (sample)	Dependencies
E8	Cutover & Decommission	4–5	EM + SRE	Progressive ramp orchestration, guardrail dashboards, runbooks, customer-support FAQ, vendor write disable, contract termination	E6, E7

Story-level rules - Stories sized in days (1, 2, 3). Anything > 3d is split. - Each story has explicit acceptance criteria and observability hook (metric / log / trace) defined in the story. - Cross-team stories (e.g., FE consuming a new admin endpoint) are **paired stories** with shared Definition of Done.

4b. Agile Development Practices

Cadence & ceremonies | Ceremony | Cadence | Owner | Output | |---|---|---|---| | Sprint planning | Bi-weekly Mon | EM + Tech Lead | Committed sprint backlog with acceptance criteria | | Daily stand-up | Daily 15 min | Tech Lead | Blockers list | | Backlog refinement | Weekly 30 min | EM | Stories sized & ready for next sprint | | Sprint review / demo | Bi-weekly Fri | EM | Stakeholder demo + recording | | Retrospective | Bi-weekly Fri | EM | 3 keep / 3 try / 3 stop actions, owners assigned | | Architecture review | Weekly 30 min | Architect | ADRs ratified, design risks logged |

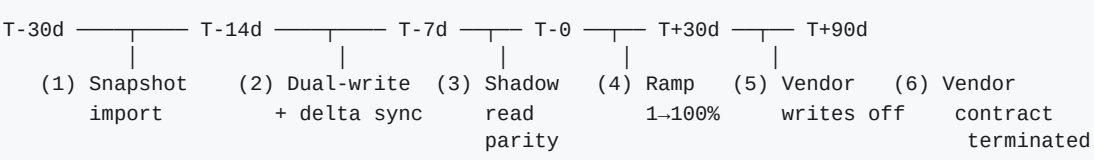
Engineering practices - **Branching**: trunk-based; short-lived branches (< 24h) merged to `main`; release happens from `main` behind feature flags. - **Code review**: minimum 1 approver, 2 for migration / cutover code; review SLA 4 working hours. - **Definition of Ready** (DoR): user story has acceptance criteria, design notes if needed, observability requirement, no unresolved blockers. - **Definition of Done** (DoD): code merged, unit tests ≥ 80%, integration test (Testcontainers) for new behavior, OpenAPI spec updated, dashboards/alerts in place, runbook entry if operational impact, demoed in sprint review. - **Quality gates** (CI): compile + unit + integration (Testcontainers PG/Kafka/Redis/OpenSearch) + contract test (Pact) + static analysis (SpotBugs, Error Prone) + dependency vulnerability scan (Trivy) + license scan; gate on PR. - **Performance gates**: weekly load test in pre-prod via k6 (5× peak); regressions block release. - **Chaos drills**: bi-weekly in pre-prod from Phase 2 (kill Redis, kill an AZ, throttle vendor); pre-cutover full game-day in week 17. - **Feature flags**: every user-visible change behind a flag; flags have explicit owner and removal date. - **On-call**: secondary rotation forms in Phase 2; primary on-call for the new service starts in Phase 3.

Metrics tracked sprint-over-sprint - Velocity, committed-vs-completed, escaped defects, P99 latency on staging, test coverage delta, flag-debt count (flags older than 90 days).

4c. System Compatibility & Data Migration

The cornerstone constraint: **the system must remain up and running throughout migration**. The strategy is staged, reversible at every step, and gated by reconciliation evidence — not by calendar dates.

Migration phases (executes in epic E7 + E8)



Step	What it does	Validation gate
1. Snapshot import	Bulk export from vendor → transform → upsert with `source=migrated_vendor`, preserving `legacy_id`	Row count matches export ± 0; sample diff < 0.01%
2. Dual-write	Every new write goes through `vendor-adapter` to **both** vendor and `review-service`	Write success rates within 0.05% of each other for 7 days
3. Shadow read	Reads hit both backends; vendor's response is returned to client; new system's response is logged + diffed	Diff rate < 0.01% on a sliding 24h window

Step	What it does	Validation gate
4. Progressive ramp	New system becomes primary for a growing % of traffic via feature flag	See guardrails below
5. Vendor writes off	Adapter switches to `new_only` mode; vendor becomes read-only / archive	Reconciliation report signed off by Product + Data
6. Decommission	Vendor contract terminated; vendor data archived to S3 for 90 days	Vendor invoice = $0

Reconciliation strategy

- **Row-level checksum**: nightly job compares `legacy_id`, `rating`, `status`, `body` hash for every migrated review. Mismatches feed an exceptions queue, triaged daily during Phase 3.
- **Aggregate parity**: per-product `avg_rating` and `count` compared between systems; alert if delta > 0.01.
- **Behavioral parity**: shadow-read diff logger captures any field-level mismatch between vendor and new-system responses, bucketed by endpoint.
- **Sign-off artifact**: `RECONCILIATION_REPORT.md` checked into the repo before cutover, listing all checks and their pass/fail status.

Progressive Rollout & Guardrails (canary, not A/B)

Goal: replace the vendor safely. We are not running a behavioral A/B; we are running a guardrailed ramp with auto-rollback. Behavioral A/B testing is reserved for **Phase 2** additive features (ranking, AI summaries).

Cohort assignment - Sticky bucketing by `hash(user_id) % 1000`. Same user always lands on the same backend within a ramp window — prevents the "I just submitted a review and now it's gone" UX. - Anonymous traffic bucketed by `hash(session_id)`. - Override flags for canary tenants (internal employees, dogfood account) so issues are spotted before customer impact.

Ramp schedule

Step	% of traffic on new system	Hold duration	Promotion criteria
0	0% (control baseline)	24 h	Baseline guardrail metrics captured
1	1% (internal + dogfood)	24 h	All guardrails green
2	5%	24 h	All guardrails green
3	25%	48 h	All guardrails green; spans a peak-traffic window
4	50%	48 h	All guardrails green
5	100%	7 days hypercare	Zero P1 incidents

A human gate (EM + on-call) must sign off each promotion. Promotion is **not automatic upward**; rollback **is** automatic downward.

Guardrail metrics & thresholds (measured per cohort, 5-min rolling window)

Metric	Threshold	Source	Action on breach
API error rate (5xx)	> 0.1%	Gateway logs	Halt ramp; if sustained 10 min → auto-rollback
Read P99 latency	> 75 ms	Micrometer	Halt ramp; if sustained 10 min → auto-rollback
Write success rate	< 99.9%	review-service metric	Halt ramp; if sustained 5 min → auto-rollback
Shadow-read diff rate (parity)	> 0.05%	diff logger	Halt ramp; investigate before continuing

Metric	Threshold	Source	Action on breach
Moderation queue lag	> 1 hour	moderation-service	Page on-call; do not auto-rollback
Conversion rate on PDP	drop > 2% vs. control	Analytics	Halt ramp; investigate
Review submission rate	drop > 5% vs. control	Analytics	Halt ramp; investigate
SEO impressions (review-rich snippets)	drop > 5% week-over-week	Search Console	Halt ramp; daily review

Auto-rollback mechanism - Implemented as a small **flag-controller** (Spring Boot CronJob, runs every 60s) that reads guardrail metrics from Prometheus and writes the flag value via the LaunchDarkly API. - One sustained breach (≥ threshold for the configured duration) → flag percentage drops by one ramp step. - Two sustained breaches in a 30-min window → flag percentage drops to **0%** (full rollback). - Every flag change is announced in `#review-cutover` Slack with the triggering metric snapshot.

Manual rollback runbook - One-button rollback documented in the cutover runbook: a single LaunchDarkly toggle (`review.new_system.percentage = 0`) reverts traffic instantly. RTO target: < 60 seconds from decision to flip. - Database is unaffected by rollback (we kept dual-write through Step 4); vendor remains the system of record until Step 5.

Cross-references - Go/No-Go gates that consume these guardrails: see **§5 Delivery**. - Feature-flag plumbing, blue/green at the service tier, and canary infrastructure: see **§7 Deployment**.

---

5. Delivery

Delivery is **continuous, behind feature flags**. The cutover itself is a flag flip, not a deploy. There are three formal delivery gates, each with explicit, auditable criteria.

5.1 Release Model

- Every sprint releases to production behind feature flags. Production code is always deployable; *visibility* is gated, not *deployment*.
- Trunk-based development; releases happen from `main`.
- Versioning: semantic on the public API (`v1`); internal services are continuously versioned by commit SHA.
- Deprecations on the legacy vendor URL adapter are announced with a sunset date; communicated per **§1c**.

5.2 Delivery Gates

Three Go/No-Go gates govern progress. EM is accountable; sign-offs collected in writing (Confluence + Slack).

Gate G1 — Build Complete (end of Phase 2)

Audience: Staff Engineer + EM + QA Lead | Criterion | Target | |---|---| | Feature parity with vendor (functional) | 100% per parity matrix | | Unit test coverage | ≥ 80% line, ≥ 70% branch | | Integration tests (Testcontainers) | All critical flows green | | Contract tests (Pact) | Vendor adapter ↔ downstream consumers green | | OpenAPI spec | Published, reviewed, no breaking diffs vs. vendor | | Threat model | Reviewed by Security; criticals resolved | | Load test in pre-prod | Sustains 5× peak for 30 min, P99 within SLO | | Observability | Dashboards live, alerts wired, runbooks linked |

Gate G2 — Migration Ready (end of Phase 3)

Audience: EM + Director + Data Eng + Product | Criterion | Target | |---|---| | Bulk import complete | Row count parity ± 0; checksum mismatches < 0.01% | | Dual-write live | 7 consecutive days, write success delta < 0.05% | | Shadow-read parity | Diff rate < 0.01% on 24h sliding window | | `RECONCILIATION_REPORT.md` | Signed by Data Eng, Product, EM | | Cutover runbook | Reviewed in tabletop exercise; rollback rehearsed | | Customer Support | FAQ published, escalation path tested | | Downstream consumers | Notified at T-30d, T-14d; integrations validated against adapter |

Gate G3 — Cutover Authorization (T-1d)

Audience: EM + Director + Executive Sponsor + SRE on-call | Criterion | Target | |---|---| | All G2 criteria still green | Yes | | **§4c guardrails baseline** | Captured for control cohort over 24h | | On-call rotation | Confirmed; war-room scheduled | | Vendor account manager | Notified; no contractual surprises | | Rollback path | Tested in pre-prod within last 7 days; RTO < 60s confirmed | | Decision | **GO / NO-GO** recorded with timestamp + signatures |

5.3 Per-Step Promotion Criteria During Ramp

Each ramp step in **§4c** (1% → 5% → 25% → 50% → 100%) is itself a delivery gate. Promotion requires: - All guardrail metrics green for the full hold duration. - No P1 / P2 incidents opened against the new system. - EM + SRE on-call sign-off in `#review-cutover`.

Demotion (rollback) is automatic and does not require sign-off — it is *recorded*, not *approved*.

5.4 Definition of "Delivered"

The project is considered **delivered** when **all** of the following hold for **30 consecutive days post-cutover**:

Condition	Owner
100% of read + write traffic on new system; vendor in archive mode	EM
All success metrics (see **§8**) within target	PM
Zero P1 incidents attributable to the new system	SRE
Vendor invoice = $0; contract terminated or in archive-only renewal	EM + Legal
Retro published; action items assigned with owners and due dates	EM
Phase 2 backlog groomed and prioritized	PM + Architect

5.5 Rollback & Recovery Posture

- **In-ramp rollback**: instant flag flip to 0%, RTO < 60s. Database state unaffected (dual-write still active through Step 4 of §4c).
- **Post-cutover rollback (Steps 5–6 of §4c)**: vendor remains read-capable for 30 days as an archive; emergency restoration would require re-enabling vendor writes (4-hour playbook documented).
- **Data recovery**: PG PITR (5-min RPO) + Kafka topic replay; vendor snapshot retained in S3 for 90 days.
- **Decision authority during incident**: SRE on-call may auto-rollback without approval; any roll-forward decision requires EM + Director.

---

6. Communication

Communication is the connective tissue that keeps the project on track and stakeholders aligned. The *plan* lives in **§1c**; this section covers *how that plan executes* across the three critical moments: during the build, during cutover, and post-launch.

6.1 During Build (Phases 1–3)

Moment	Action	Owner	Channel
Sprint start	Share sprint goal + top 3 risks in writing	EM	`#review-revamp` Slack + Confluence
Mid-sprint blocker	Raise immediately, do not wait for stand-up	Engineer	`#review-revamp` Slack @EM

Moment	Action	Owner	Channel
Weekly status (every Friday)	RAG status email: Red/Amber/Green on scope, timeline, quality, risks; 3 bullets max per section	EM	Email to exec sponsor + Confluence
ADR ratified	Post summary + link to `docs/adr/` in `#review-revamp`	Author	Slack
External API consumer notice (T-30d)	Send deprecation notice + migration guide for vendor URL adapter	TPM	Email + dev-portal
External API consumer notice (T-14d)	Follow-up with deadline and support contact	TPM	Email
Bi-weekly stakeholder demo	Live demo of working software; record for async viewers	EM	Zoom; recording in Confluence
Risk register changes	Flag any new High-impact risk within 24h of identification	EM	Slack DM to exec sponsor + update Confluence doc

Principles during build - Bad news travels fast and upward: surface blockers and risk changes within 24 hours — never sit on them until the weekly report. - Decisions are documented in ADRs the day they are made, not reconstructed later. - Status emails lead with the RAG colour; the exec sponsor should not have to read past the first paragraph to know if they need to act.

6.2 During Cutover (Phase 4 — ramp week)

Moment	Action	Owner	Channel
T-7d	Cutover brief to all stakeholders: ramp schedule, guardrail thresholds, rollback runbook link, CS FAQ link	EM	Email + 30-min Zoom
T-1d	Go/No-Go decision recorded + announced	EM + Director	`#review-cutover` Slack + email to sponsor
Ramp start	War-room opens	SRE on-call	`#review-cutover` Slack (pinned topic = current ramp %)
Every ramp step (1%→5%→25%→50%→100%)	Guardrail snapshot posted: metric values vs. thresholds, go/no-go for next step	SRE on-call	`#review-cutover` Slack
Every 2 hours during ramp	Executive update: current %, key metrics, next step ETA	EM	`#review-exec` Slack
Auto-rollback trigger	Immediate alert with metric that breached, action taken, ETA for resolution	Flag-controller → PagerDuty → Slack	`#review-cutover` + `#review-exec`
Manual rollback decision	Record decision, metric evidence, and who made the call	EM	`#review-cutover` + Confluence incident log
100% cutover reached	Announce success; thank team; summarise metrics vs. targets	EM	`#review-revamp` Slack + email to all stakeholders
Vendor writes disabled	Confirm to vendor account manager; retain archive SLA in writing	EM	Email

War-room hygiene - One person owns the `#review-cutover` channel at all times (rotating every 4 hours during extended ramps). - All decisions posted as threaded messages with timestamp + decision-maker name; no verbal-only

decisions. - Non-essential commentary in a separate `#review-cutover-banter` channel to keep the main channel signal-only.

6.3 Post-Launch (Phase 5 — hypercare)

Moment	Action	Owner	Channel
Day 1 post-cutover	Metrics snapshot vs. baseline sent to all stakeholders	EM + PM	Email
Daily for first 7 days	Brief metrics update: error rate, P99, review submission volume, moderation queue depth	SRE on-call	`#review-revamp` Slack
Weekly for 4 weeks	Full KPI readout vs. targets (see §8); any open P2+ incidents	EM + PM	Email + Confluence
CS escalations	Triage within 2 hours; root-cause posted to `#review-revamp`; customer response within 24h	On-call BE	PagerDuty → Slack
External consumer notice (T+30d)	Vendor adapter sunset reminder; 60-day window before adapter removed	TPM	Email + dev-portal
External consumer notice (T+60d)	Final notice: adapter removed in 30 days	TPM	Email
T+30d	Written project retro published; 3 keep / 3 improve / 3 action items with owners + due dates	EM	Confluence; shared with exec sponsor
T+30d	Phase 2 roadmap briefing: what's next (AI summaries, multilingual, A/B framework)	EM + PM	30-min stakeholder Zoom
Hypercare end declaration	Formal handoff to steady-state on-call; project Slack channels archived	EM	Email + Slack

6.4 Communication Anti-patterns to Avoid

Anti-pattern	Why it hurts	What to do instead
Surprises in the weekly email	Breaks trust with the exec sponsor	Surface risks within 24h of identification
Verbal decisions in war-room	No audit trail; disputed later	Post every decision as a threaded message
Jargon-heavy updates to Director / Sponsor	Obscures real risk	Lead with business impact, follow with technical detail
Over-communicating in a low-signal channel	Desensitises the team to alerts	Keep `#review-cutover` signal-only during ramp
Waiting for all answers before raising a blocker	Delays resolution	Raise with partial information; refine as you go

---

7. Deployment

7.1 Deployment Strategy

Service-level: Blue/Green

Each Spring Boot service (`review-service` , `moderation-service` , `vendor-adapter` , `migration-worker`) is deployed blue/green on Kubernetes:

- Two identical Deployments (`-blue` , `-green`) exist behind a Service selector.

- A new version is deployed to the inactive slot; smoke tests run against it.
- Traffic flips atomically by updating the Service selector label.
- Rollback is an identical selector flip; RTO < 30 seconds.
- Both slots run simultaneously for at most 15 minutes during a deploy; zero dropped connections via `preStop` lifecycle hook + graceful shutdown (`server.shutdown=graceful` in Spring Boot).

Traffic-level: Canary via Feature Flags

Within the running service, traffic is split by feature flag (LaunchDarkly / OpenFeature SDK) as described in **§4c**. This is *orthogonal* to blue/green — it controls *which backend* handles a request, not *which version of the service*.

Database migrations: Expand-Contract

All schema changes follow the expand-contract pattern to keep migrations non-breaking across deploys:

Step	Action	Deploy
Expand	Add new column / table (nullable or with default)	Deploy N
Migrate	Backfill data; update application code to write both old + new	Deploy N+1
Contract	Drop old column / table once no code reads it	Deploy N+2 (safe to run weeks later)

Rules: never rename a column in a single deploy; never add a NOT NULL column without a default; Flyway migration must be backwards-compatible with the previous application version.

7.2 Environment Topology

Environment	Purpose	Promotion gate
`dev`	Individual feature branches; ephemeral per PR (spin-up + teardown via CI)	PR opened
`staging`	Full integration environment, mirrors prod topology; Testcontainers replaced by real Kafka/PG/Redis/OpenSearch	Sprint review demo; load test gate
`pre-prod`	Production clone with anonymised data; chaos drills run here; load tests at 5× peak	Gate G1 + G2 (see §5.2)
`prod`	Live system	Gate G3 + progressive ramp

Staging and pre-prod use the same Helm charts as prod; only image tags and secret references differ.

7.3 CI/CD Pipeline

```
PR opened
   │
   ├── compile + unit tests (JUnit 5)
   ├── integration tests (Testcontainers: PG, Kafka, Redis, OpenSearch)
   ├── contract tests (Pact broker)
   ├── static analysis (SpotBugs, Error Prone, Checkstyle)
   ├── dependency vulnerability scan (Trivy)
   └── build Docker image → push to registry with commit SHA tag

Merge to main
   │
   ├── all above gates (re-run)
   ├── deploy to staging (blue/green)
   ├── smoke test suite (critical paths: submit, read, moderate)
   └── publish OpenAPI spec diff (blocking on breaking changes)

Weekly (scheduled)
   │
   ├── load test in pre-prod (k6, 5× peak, 30 min)
   ├── dependency update PR (Renovate bot)
   └── chaos drill (kill Redis / kill AZ / throttle vendor adapter)

Release to prod
   │
   ├── Gate G3 sign-off (§5.2)
   ├── deploy to prod inactive slot (blue/green)
   ├── smoke test against inactive slot
   ├── flip Service selector
   └── progressive ramp via feature flag (§4c)
```

All pipeline runs are tracked in GitHub Actions; failures page the owning team, not a shared ops queue.

7.4 Observability

Metrics (Micrometer → Prometheus → Grafana)

Dashboard	Key signals
Service RED	Request rate, error rate (4xx/5xx split), P50/P95/P99 latency — per endpoint
Business KPIs	Reviews submitted/day, approval rate, moderation SLA p50/p95, helpful-vote rate, submission success rate
Infrastructure	JVM heap, GC pause, virtual-thread count, HikariCP pool wait, Kafka consumer lag, Redis hit rate, OpenSearch indexing lag
Migration	Dual-write divergence rate, shadow-read diff rate, reconciliation mismatches, ramp %
Cutover guardrails	All §4c guardrail metrics on a single board; threshold lines drawn; auto-rollback events marked

Tracing (OpenTelemetry → Jaeger / Tempo) - Trace context propagated end-to-end: HTTP headers → Kafka message headers → DB spans. - Sampling: 100% for errors, 1% for healthy traffic in prod; 100% in staging. - Trace IDs included in all structured log lines and RFC 7807 error responses.

Logging (structured JSON → ELK / Loki) - Every log line includes: `traceId`, `spanId`, `service`, `env`, `userId` (hashed), `productId`, `severity`. - No PII in logs (user name, email, review body truncated; flagged by a log-linting CI check). - Log levels: ERROR for actionable failures; WARN for degraded-mode events; INFO for business events (review submitted, approved, rejected); DEBUG off in prod.

Alerting | Alert | Threshold | Severity | Paging | |---|---|---|---| | API error rate | > 0.5% sustained 5 min | P1 | PagerDuty on-call | | Read P99 latency | > 100 ms sustained 5 min | P2 | PagerDuty on-call | | Kafka consumer lag (`moderation-service`) | > 10k messages | P2 | PagerDuty on-call | | Dual-write divergence | > 0.05% | P2 | Slack `#review-ops` | |

JVM heap > 85% | Sustained 10 min | P2 | Slack `#review-ops` | | Flyway migration failed | Any | P1 | PagerDuty on-call | | Certificate expiry | < 14 days | P3 | Slack `#review-ops` |

7.5 Security & Compliance Controls

- **Auth**: OAuth2 JWT validated at API Gateway (Kong); service-mesh mTLS for inter-service calls.
- **Secrets**: HashiCorp Vault; injected at runtime via Vault Agent sidecar; never in environment variables or config maps.
- **Image security**: base image pinned to digest; Trivy scan on every build; no `latest` tags in prod.
- **RBAC**: K8s namespaced RBAC; `review-service` ServiceAccount has least-privilege PG credentials (no DDL in prod).
- **Network policy**: `review-service` can reach Postgres, Redis, Kafka, OpenSearch only; no egress to internet except via explicit proxy allowlist.
- **GDPR**: tombstone event triggers within 24h; Kafka compaction on `user.gdpr` topic; PG `deleted_at` soft-delete with scheduled hard-delete job.
- **Audit log**: all moderation state transitions + admin actions stored in `moderation_event` table and emitted to `audit.events` topic; retained in S3 Glacier for 7 years.

7.6 Disaster Recovery

Scenario	RPO	RTO	Mechanism
Single pod crash	0	< 30s	K8s liveness probe + auto-restart
AZ failure	0	< 2 min	Multi-AZ K8s node groups; PG Aurora multi-AZ automatic failover
Bad deploy	0	< 30s	Blue/green selector flip
Kafka topic corruption	5 min	30 min	Kafka consumer replay from last committed offset + S3 backup
Postgres data loss	5 min	30 min	Aurora PITR to 5-min granularity
Redis total loss	0 (cache only)	< 5 min	Services degrade gracefully to PG replica; Redis auto-restores from replica
Full region failure	60 min	4 hr	Read-only mode from DR region (S3-backed static responses for PDP); active-active is Phase 2

DR playbooks for each scenario are maintained in Confluence and reviewed quarterly.

---

8. Post-Release Success

8.1 Success Metrics

Success is defined as **measurable improvement on the legacy bottlenecks** (§2.1) without regression on user-facing or business KPIs. Each metric has a baseline (captured pre-cutover), a target, and a measurement window.

Tier 1 — Business KPIs (Director / Sponsor view)

Metric	Baseline	Target (T+30d)	Stretch (T+90d)	Source
Total infra + vendor cost	$X / month (vendor)	≤ 60% of baseline	≤ 50% of baseline	Finance
PDP conversion rate	(current value)	Flat or up	+1%	Analytics

Metric	Baseline	Target (T+30d)	Stretch (T+90d)	Source
Review submission rate (per 1k PDP views)	(current value)	Flat or up	+5%	Analytics
SEO impressions on review-rich snippets	(current value)	Flat or up	+5%	Search Console
GDPR delete turnaround	7+ days	< 24 hours	< 6 hours	Audit log
Moderation SLA (median time to decision)	(current vendor)	< 4 hours	< 2 hours	moderation-service metric

Tier 2 — Technical KPIs (Staff / SRE view)

Metric	Baseline	Target (T+30d)	Source
Read P99 latency (PDP rating + listing)	~250 ms (vendor sync call)	< 50 ms	Micrometer / Grafana
Write P99 latency (submit review)	(current)	< 200 ms	Micrometer
Read availability	~99.5% (vendor)	≥ 99.95%	Synthetic probes
Submission success rate	(current)	≥ 99.9%	review-service metric
Shadow-read parity at cutover	n/a	≥ 99.99%	Reconciliation report
Spam / abuse leakage rate	(current — manual)	< 0.5% of approved reviews	Sample audit

Tier 3 — Project Health (EM view)

Metric	Target
P1 incidents in hypercare (T+0 to T+30d)	0
P2 incidents in hypercare	≤ 2, all resolved within SLA
Auto-rollback events during ramp	≤ 1 per ramp step
Post-launch hot-fixes	≤ 3 in first 30 days
Vendor invoice at T+30d	$0
Team-reported burnout (sprint retro)	No "burnt out" or "unsustainable" flags

8.2 Measurement Cadence

Window	What's measured	Audience	Format
Daily (T+0 to T+7d)	Tier 2 + Tier 3 metrics; any open incidents	EM, SRE, Director	Slack `#review-revamp` snapshot
Weekly (T+7d to T+30d)	All three tiers vs. targets; flag any miss	EM, Director, Sponsor	Email + Confluence dashboard link
Monthly (T+30d onward)	Tier 1 + Tier 2 trends; cost actuals; Phase 2 progress	Director, Sponsor	Email + monthly business review
Quarterly	Architectural review: are SLOs still right? Are bottlenecks reappearing?	Architect, EM, SRE	1h architecture forum

A **single Grafana dashboard** ("Review System — Post-Launch") aggregates Tier 1 + Tier 2 metrics with baseline lines, target lines, and a status indicator per row. Director can self-serve without waiting for the weekly email.

8.3 Hypercare Plan (T+0 to T+30d)

Element	Detail
Duration	30 days from 100% cutover
On-call	Primary + secondary, drawn from the project team (not steady-state ops yet)
Response SLA	P1: 15 min · P2: 1 hr · P3: next business day
Daily triage	15-min stand-up reviewing open issues, error-rate trends, queue depth
War-room reactivation	If 2+ P2 incidents in 24h or 1 P1, war-room reopens until stable for 24h
Rollback authority	EM can authorise full rollback (re-enable vendor) within hypercare window; requires Director sign-off after T+7d
Hypercare exit	T+30d if all Tier 1 + Tier 2 metrics within target and zero open P1; otherwise extend by 2 weeks

8.4 Retrospective

A formal retro is held at **T+30d**, written up and shared with the exec sponsor. Format:

- **What worked well** (3 items, with evidence)
- **What didn't work** (3 items, with evidence)
- **Action items** (3–5, each with owner + due date + success measure)
- **Decisions to revisit** (ADRs that didn't pan out as expected)
- **Praise** (named contributions worth celebrating)

Action items are tracked in the Phase 2 backlog with the `retro-action` label. The retro doc itself is linked from the project README so future projects can learn from it.

8.5 Phase 2 Roadmap (Post-Hypercare)

Phase 2 starts only after hypercare exit. Candidate epics, prioritised in the retro:

Epic	Why now	Effort estimate
AI-generated review summaries on PDP	Unlocked by owned data; high user value; LLM cost manageable at our scale	6 weeks, 2 BE + 0.5 ML
Multilingual moderation (top 5 languages)	International expansion ask from PM	4 weeks, 1 BE + ML model swap
Question & Answer module	Frequently requested adjacent feature	8 weeks, 2 BE + 1 FE
A/B testing framework	Needed for behavioural experiments on ranking, PDP layout	4 weeks, 1 BE + 0.5 Data Eng
Active-active multi-region	Reduces RTO from 4h to < 5 min	12 weeks, 1 SRE + 1 BE
Vendor adapter sunset	Remove transitional code; reclaim ownership clarity	2 weeks, 1 BE

Phase 2 is sequenced based on business value × confidence; the AI summary epic is the leading candidate because the data foundation is now in-house.

8.6 Long-Term Health Checks

Beyond Phase 2, the system needs ongoing stewardship to avoid becoming the next legacy:

- **Quarterly architectural review**: revisit ADRs, check for drift, retire flags older than 90 days, rotate dependencies.
- **Annual SLO review**: are targets still appropriate as traffic grows? Tighten or relax based on data.
- **Annual cost review**: validate the 60% saving holds; right-size reserved instances.

- **Annual chaos game-day**: full multi-failure simulation with the on-call team.
- **Documentation freshness**: ADRs and runbooks reviewed every 6 months; staleness gated in CI.

The goal is that the system this project built remains **easy to evolve** — the success of the rebuild is measured not just at T+30d, but at T+2 years, when the next team finds it a pleasure rather than a burden to work in.